FINISHING UPGRADES

De Rueda Construction — Simplification, UI/UX & Payment Setup Specification

Purpose. Finish the existing system by preserving useful features, removing unnecessary complexity, and making everyday workflows clear for the CEO, HR, site supervisors, canteen staff, and customer deployments. This is a specification and checklist—not a claim that code has already been changed or deployed.

PRIORITY RULE: keep the system straightforward. Do not add duplicate dashboards, overlapping payment hubs, repeated approvals, or infrastructure without a real business need.

1. Simplified product scope

Area	Required outcome
Dashboard	One useful overview: key workforce/site figures, pending actions, alerts, and recent activity.
Employees & HR	Employee records, stable employee IDs, site assignment, attendance, and necessary documents in a clear workflow.
Payroll	Calculate gross pay, configured deductions, and net pay; preserve payroll history and produce the reports actually used.
Canteen	GCash-only canteen checkout. Each site has its own editable GCash destination settings, with appropriate access and CEO oversight.
Customer payments	Use PayMongo for customer invoice collection if required. Do not mix customer payments with payroll or worker payouts.
Sites & projects	Site-specific workers, attendance, canteen configuration, materials/expenses where already needed, and reports.
Reports	Keep necessary PDF/Excel exports; show selected filters and generation time, and make output practical to print.
Storage	One authoritative data source and a dependable backup/restore process. Avoid competing copies of the same record.

2. Highlighted UI simplification & brand direction

Keep the supplied De Rueda UI mockups as the layout reference unless a specific change is requested. Apply the preferred glossy lime-green De Rueda wordmark/emblem consistently. Use green, dark blue, light blue, white, and black; pink-glitter styling is a tiny decorative accent only.

• Use one consistent app shell: desktop sidebar; collapsible navigation on smaller screens; clear page title; one primary action; useful search/filters; consistent tables/cards; simple dialogs and status feedback.

• Remove duplicate menu items and overlapping screens. Group related functions under the most natural section instead of creating a new module for every small task.

• Make mobile layouts fit the screen: collapse navigation, stack summary cards, allow table row details, avoid horizontal overflow, and keep important buttons reachable.

• Provide visible keyboard focus, correctly associated form labels, readable contrast, helpful empty/loading/error/success states, and clear validation messages.

• Keep all features honest: every button should work as labelled or be marked unavailable; never show mock data or a simulated transaction as a real result.

3. Highlighted Messages / chatbox specification

DESIGN INTENT: make the internal company chat easy to use like a familiar modern messenger, without turning it into a separate complicated product.

Part	Required behaviour
Desktop layout	Three clear areas: conversation list/search on the left, active chat in the center, and conversation details/participants on the right. Let the center chat remain the main focus.
Mobile layout	Use a simple two-step flow: conversation list → selected conversation. Hide the side details panel behind a details button/drawer; do not squeeze three columns onto a phone.
Composer	A readable message field with Send, attachment button, and support for photos/files. Multiline typing should work; Enter/Shift+Enter behaviour must be consistent and explained.
Attachments	Show selected filename/preview where possible, upload progress, success/failure status, and retry. Validate file type/size and enforce access on the server.
Messages	Show sender, timestamp, message content, and attachment state clearly. Conversation selection should load the correct messages; avoid fake sent/saved states.
Access	Only authenticated company users with permission to the relevant conversation/site can read or send. Do not promise encryption unless it is implemented and verified.

Chatbox acceptance test: open a conversation, send a message, refresh and confirm it remains; switch conversations and verify the correct history; attach a photo/file; test upload failure and retry; confirm unauthorized users cannot access restricted conversations; test phone width with no horizontal overflow.

4. Canteen GCash by construction site

KEEP THIS WORKFLOW SMALL: a site selector, the site's current GCash destination details, an edit/save action for authorized users, and a short change history. Do not create another GCash hub just for this.

• Canteen checkout offers GCash only; do not display cash, card, or unrelated e-wallet choices in that flow.

• Store destination settings per site. Changing Site A must not change Site B. The checkout must bind to the selected site's saved settings, not trust an arbitrary client-supplied destination.

• Allow authorized site staff to edit only their assigned site's destination; CEO can view/oversee all sites. Record who changed the details and when.

• Do not silently redirect an already-created pending payment when a destination is changed. Expire or regenerate the request clearly.

• A payment screenshot or tapping a button is not proof of payment. Mark paid only after a reliable provider confirmation or an explicitly defined, controlled manual verification process.

Settlement warning: PayMongo's GCash checkout option does not automatically mean payment settles directly into every site's own GCash wallet. Verify the customer's actual merchant/settlement arrangement before enabling the flow.

5. PayMongo integration & customer setup

Use PayMongo Hosted Checkout for customer invoices only if the business needs it. Keep payment initiation and confirmation on the server, verify signed webhooks, safely handle repeated events, and reconcile payment status. Do not confirm a payment based only on a return/success URL. Keep customer collections separate from canteen collection and payroll/disbursement features.

Highlighted requirement — keep the environment-variable dialog like the one shown in the supplied photo, so setup can be completed later when onboarding each customer.

Variable name	What to enter / how it is used
`PAYMONGO_SECRET_KEY`	The customer's private PayMongo secret key. Server-side only; never expose in browser code.
`PAYMONGO_PUBLIC_KEY`	The customer's public key for supported client-side checkout use. Do not treat it as a substitute for server-side validation.
`PAYMONGO_WEBHOOK_SECRET`	The webhook-signing secret/configuration used by the server to verify provider event authenticity.

Variable name	What to enter / how it is used
`PAYMONGO_ENVIRONMENT`	Environment selector: test first; use live only after successful verification and customer authorization.

Configuration behaviour

• Keep these four fields in the same environment-variable dialog; values should be masked by default and editable only by an authorized deployer.

• Allow customer-specific configuration so one customer's credentials cannot affect another customer's deployment. Never use one customer's keys as defaults for all customers.

• If a required value is missing or invalid, show a clear setup/error state and do not pretend checkout is active.

• Keep secret keys and webhook secrets in server-side environment configuration or a secrets manager, not in frontend code, localStorage, public files, or ordinary logs.

• Use test credentials and test events first; switch to live mode only after the customer's authorized credentials, webhook verification, and end-to-end payment tests pass.

• The app can provide the fields and validation, but it cannot invent PayMongo credentials. Obtain them from the customer's authorized PayMongo account.

6. Security and audit items to fix before production

• Remove passwords, MFA secrets, backup codes, and private employee/payment records from frontend-delivered bundles. Rotate any real credentials that were exposed.

• Enforce authentication and role/site authorization server-side for every sensitive API, export, file, and payment operation. Hiding a button is not security.

• Use a reliable authoritative data store; browser localStorage must not be the sole source of truth for payroll, employees, payment status, or audit records.

• Protect hiring/inquiry forms with server-side validation, appropriate privacy notice/consent and retention rules; never let fallback form submission put personal data into a URL.

• Fix API routes that return homepage HTML instead of their expected JSON, and ensure robots.txt, sitemap.xml, security.txt, manifest and favicon return correct content types.

• Add appropriate security headers, keyboard focus, label associations, skip/main landmarks, and fix mobile navigation overflow.

7. Final implementation order

P0 — Stop critical risk

Remove exposed secrets/private records, rotate exposed credentials, secure sensitive APIs, and remove fake successful GCash/payment transactions.

P1 — Simplify the UI

Consolidate navigation/settings, preserve the agreed layout references, improve phone/tablet responsiveness, and complete the chatbox usability requirements.

P2 — Complete payment setup

Add the reusable PayMongo environment dialog, customer-specific configuration, test/live selection, server-side webhook verification, and truthful payment states.

P3 — Finish site workflows

Implement GCash-only canteen checkout with independent per-site destinations, permissions, and clear confirmation rules.

P4 — Data, privacy & reports

Set authoritative storage/backups, secure forms, preserve history, and validate the reports people actually need.

P5 — Verify & deploy

Run the build, tests, desktop/mobile checks, and deployed-site verification.

8. Required acceptance checklist

■ Use npm only; synchronize package.json and package-lock.json; remove conflicting Bun lockfiles.

■ Install dependencies and fix TypeScript/dependency errors; npm run build must pass before deployment readiness is claimed.

■ Verify the published site is the intended latest build; test routes, refresh, forms, assets, and key journeys on desktop and mobile.

■ Test role/site access, canteen site-specific destination changes, payment success/failure, invalid/duplicate webhooks, timeouts, and unauthorized invoice access.

■ Verify that no secrets or private employee/payment records appear in the deployed JavaScript bundle.

■ Confirm backups and restoration procedure; label mock/sandbox/unavailable features honestly.

Definition of done: a clean, understandable interface; useful existing business features preserved; customer-specific PayMongo setup ready for authorized configuration; site-specific GCash canteen rules; real payment confirmations only; no exposed secrets; and a successful npm run build plus deployment verification.